Resumen de operadores por: Rocha Vazquez Jarvis Gael

Tarea #08

1. Operadores Aritméticos

Son los más cercanos a las matemáticas tradicionales. Realizan cálculos matemáticos básicos y complejos.

- **Suma (+) y Resta (-)**: Realizan adiciones y sustracciones.
- **Multiplicación (*) y División (/)**: Multiplican o dividen valores. En muchos lenguajes, la división tradicional devuelve números decimales (flotantes).
- **División Entera (//)**: Realiza una división, pero descarta los decimales, quedándose solo con el número entero.
- **Módulo (%)**: Este es muy especial en programación; devuelve el residuo de una división. Es muy útil para saber si un número es par o impar.
- **Exponente (**):** Eleva un número a la potencia de otro.

Ejemplos de expresiones:

- resultado = 10 + 5 (El resultado es 15)
- residuo = 10 % 3 (El residuo es 1, porque 3 cabe 3 veces en 10, y sobra 1)
- área = lado ** 2 (Eleva el valor de 'lado' al cuadrado)

2. Operadores Relacionales

Su propósito es establecer la relación de tamaño o posición entre dos valores. Nos ayudan a responder preguntas como "¿es este número mayor que este otro?". El resultado de estas operaciones siempre es un valor booleano: Verdadero (True) o Falso (False).

- **Mayor que (>) y Menor que (<)**
- **Mayor o igual que (>=) y Menor o igual que (<=)**

Ejemplos de expresiones:

- edad >= 18 (Devuelve True si la persona tiene 18 años o más)
- temperatura < 0 (Devuelve True si hace frío bajo cero)

3. Operadores Lógicos

Estos operadores toman decisiones combinando múltiples condiciones. Son la base del razonamiento en un programa. En lenguajes como Python se leen tal cual en inglés, mientras que en otros lenguajes usan símbolos como && o ||.

- **AND (and / &&)**: Exige que **todas** las condiciones sean verdaderas. Si una falla, todo es falso.
- **OR (or / ||)**: Es más flexible; solo requiere que **al menos una** condición sea verdadera.
- **NOT (not / !)**: Invierte el resultado. Si algo era verdadero, lo vuelve falso, y viceversa.

Ejemplos de expresiones:

- (edad >= 18) and (tiene identificación) (Ambas deben cumplirse para poder entrar a un lugar)
- (es fin de semana) or (es día festivo) (Basta con que una sea cierta para no ir a trabajar)
- not (esto lloviendo) (Es verdadero solo si NO está lloviendo)

4. Operadores de Asignación

Como su nombre indica, sirven para asignar o "guardar" un valor dentro de una variable. No debes confundirlos con los de igualdad matemática.

- **Asignación simple (=)**: Guarda el valor de la derecha en la variable de la izquierda.
- **Asignación compuesta (+=, -=, *=, /=)**: Hacen una operación matemática y luego guardan el resultado en la misma variable. Es una forma de ahorrar código.

Ejemplos de expresiones:

- puntuación = 100 (Guarda el número 100 en la variable 'puntuación')
- vidas -= 1 (Es lo mismo que escribir vidas = vidas - 1. Le resta 1 a la variable original)

5. Operadores de Comparación

Aunque a menudo se estudian junto a los relacionales, los de comparación se enfocan estrictamente en verificar si dos cosas son exactamente idénticas en su valor o si son distintas.

- **Igualdad (==)**: Verifica si el valor de la izquierda es idéntico al de la derecha. (Nota que se usan dos signos de igual, a diferencia de la asignación).
- **Desigualdad (!=)**: Verifica si los valores son diferentes.

Ejemplos de expresiones:

- Contraseña ingresada == "secreta123" (Verifica si el usuario escribió la contraseña correcta)
- estado cuenta != "suspendida" (Verifica que la cuenta NO esté suspendida)

6. Operadores de Identidad

Estos operadores son muy particulares y utilizados en lenguajes modernos. No comparan si dos variables tienen el mismo valor, sino si **son exactamente el mismo objeto en la memoria de la computadora**.

- **Es (is)**: Devuelve True si ambas variables apuntan al mismo espacio de memoria.
- **No es (is not)**: Devuelve True si apuntan a espacios diferentes.

Ejemplos de expresiones:

- Imagina que tienes dos listas idénticas: lista_a = [1, 2] y lista_b = [1, 2].

- Si haces lista_a == lista_b, el resultado es **True** (tienen los mismos números, sus valores son iguales).
- Pero si haces lista_a is lista_b, el resultado es **False** (son dos listas distintas creadas en diferentes lugares de la memoria).

La Prioridad de los Operadores (Jerarquía)

Cuando combinas muchos operadores en una sola línea de código, la computadora necesita saber qué resolver primero, al igual que en las matemáticas. A esto se le llama **precedencia o prioridad de operadores**.

Si no usas paréntesis para agrupar tu lógica (lo cual siempre es recomendable para mantener el orden), la computadora resolverá las operaciones en este orden de mayor a menor prioridad:

1. **Paréntesis ()**: Lo que está adentro siempre se resuelve primero. Es el rey de la prioridad.
2. **Exponentes (**)**: Las potencias van antes que la multiplicación.
3. **Aritméticos de multiplicación/división (*, /, //, %)**
4. **Aritméticos de suma y resta (+, -)**
5. **Relacionales y de Comparación (>, <, >=, <=, ==, !=)**: Después de resolver los números, se comparan sus tamaños.
6. **Identidad (is, is not)**
7. **Lógicos (not, luego and, y al final or)**: Se resuelven las combinaciones de verdadero/falso.
8. **Asignación (=, +=, etc.)**: Es lo último que ocurre. Una vez que toda la expresión de la derecha se ha calculado, el resultado final se guarda en la variable.

Fuente: https://articulos.euroinnova.com/innovacion-digital/que-es-un-operador-en-informatica